Chalmers | Göteborgs Universitet 2023-01-10

Examiner: David Sands, dave@chalmers.se, D&IT,
Planned visit to the examination halls approx 9 - 9.30 to answer questions. At other times by phone 0737207663

Functional Programming TDA 452, DIT 143

2023-01-10 08.30 – 12.30 Samhällsbyggnad SB

0737207663

- There are two parts to this exam. You must pass Part I to get a pass on the course. Part I has a total of 26 points; 16 points guarantees a pass on the exam (3/G).

  Part II has a total of 26 points and is graded if part I is passed. 10 points on part II guarantees a grade 4, 18 points on part II guarantees a grade 5/VG.

  The final grade boundaries may be slightly lower but will never be higher than those given. Any bonus points from 2022 will not be included in the reported ladok grade; if you think your bonus points can make a difference to your reported grade (once you receive it) then please contact the examiner in slack and he will be happy to check this for you.
- Results: usually within two weeks.
- **Permitted materials:** None (i.e. standard items only: pens, pencils, Dictionary, etc).
- **Please read the following guidelines carefully:**
  - Read through all Questions before you start working on the answers.
  - Begin each whole question on a new sheet.
  - Write clearly; unreadable = wrong! **If you give multiple answers to the same question only the first answer will be graded.**
  - For each part Question, if your solution consists of more than a few lines of Haskell code, use your common sense to decide whether to include a short comment to explain your solution.
  - You can use any of the standard Haskell functions *listed at the back of this exam document*, as well as any methods (functions) from the standard type classes Monad, Functor, or Applicative.
  - **Full points** are given to solutions which are **short**, **elegant**, and **correct**. Fewer points may be given to solutions which are unnecessarily complicated, unstructured, or repetitive.
  - You are **encouraged to use** the solution to an **earlier part** of a Question to help solve a **later part** — even if you did not succeed in solving the earlier part.

Part I

1. *(6 points)* In this question you are to define a function

   ```
   average :: Fractional a => [a] -> Maybe a
   ```

   which computes the average of a given non-empty list of fractional numbers (the class of numbers supporting real division, `(/)`). For example, `average [0.1, 0.2, 0.3, 0.4]` gives `Just 0.25`. If the input list is empty then the result should be `Nothing` (since you cannot divide by zero).

   - *(2 points)* Give a definition of average using standard functions but without recursion. Hint: `Int` is not in class `Fractional`, but can be easily converted to any other kind of number using `fromIntegral :: (Integral a,Num b) => a -> b`.
   - *(4 points)* Give a definition *without* using standard functions. Your definition should compute the average with a single traversal of the input list, by defining just a single recursive helper function. Only use standard functions `(+)` and `(/)` in your definition (you should not need `fromIntegral`). Solutions which do not follow these requirements get 0 points. Hint: your helper function will need two extra parameters.

2. *(8 points)*

   - *(2 points)* Give a data type `Arith` suitable for representing any arithmetic expressions built from floating point numbers, together with binary operators addition, subtraction, multiplication, exponentiation (a^b), max, and min.

     Your data type should *not* be able represent invalid expressions (e.g. containing operators not listed above), or expressions containing variables. You are free to define additional helper types or type synonyms as needed.
   - *(1 points)* Give the definition of the (unsimplified) arithmetic expression representing $10 + 20^{(4+5)}$
   - *(5 points)* Define a function

     ```
     showOps :: Arith -> String
     ```

     so that if `a1` is the arithmetic expression representing the expression $10 + 20^{(4+5)}$ then `putStrLn (showOps a1)` will produce the following output:

     ```
     Add: 2
     Sub: 0
     Mul: 0
     Exp: 1
     Max: 0
     Min: 0
     ```

     representing that the expression contains two additions, one exponentiation, and no other operators. I.e. `showOps` constructs a string representing a table which shows how many of each type of operator there is in the expression. Exactly how you name the operators in the table, and in which order they appear, is up to you.

3. *(6 points)* Given the following Data type

```
data T = A (Maybe Integer)| B T Integer T
  deriving Show
```

- *(2 points)* Give an example of a value of type `T` which contains exactly two Integers.
- *(4 points)* Define a function

  ```
  mapT :: (Integer -> Integer) -> T -> T
  ```

  so that, for example, if `t :: T` then `mapT (*2) t` produces a tree like `t`, except that every number in the tree is doubled.

---

4. *(6 points)* The module `System.Directory` has a function

   ```
   listDirectory :: FilePath -> IO [FilePath]
   ```

   which given the name of a directory (folder) on the system, provides a command for listing all the files in that directory. Using this function, define a function

   ```
   listDirectoryContents :: FilePath -> IO [(FilePath,Int)]
   ```

   which when run on a given input directory produces a list of pairs of the filenames and their size (measured in number of characters. Hint: use `readFile :: FilePath -> IO String` to access the contents of a file. You may assume that a directory only contains readable files, and that the input directory does exist.

Part II

5. *(4 points)* The module `Data.List` contains a function

   ```
   findIndices :: (a -> Bool) -> [a] -> [Int]
   ```

   which finds the positions of all elements for which the first argument would give `True`. For example,

   ```
   prop_FindIndicesExample =  findIndices even [10,3,5,7,2] == [0,4]
   ```

   I.e. at positions `0` and `4` in the list `[10,3,5,7,2]` there are even numbers.

   Suppose that we have a definition of `findIndices` (you should not define it). Give a quickCheck property for testing the definition of `findIndices` which checks that property `p` holds for the elements at all positions listed in the result, and not for any other positions.

6. *(8 points)* An instant messenger app contains a simple line editor, which allows users to type messages and edit them as they type; users can type normal characters, delete a character, and move the cursor (the point at which the next character will be inserted) left or right.

   The following data type models the key presses:

   ```
   data KeyPress = Alpha Char | Delete | GoLeft | GoRight | Return
     deriving (Eq,Show)
   ```

   Write a function

   ```
   lineEdit :: [KeyPress] -> String
   ```

   which converts a list of key presses into the String that would be displayed. For example

   ```
   lineEdit [Alpha 'a', Alpha 'e', Alpha 'c', GoLeft,
             Delete, Alpha 'b', Return, Delete, GoRight, Alpha 'd']
   ```

   should give the string `"abc\nd"`. Note how `Return` causes the whole current string to be put on a line, and that the cursor starts afresh from the next line. Note also in this example how the second `Delete` instruction and `GoRight` have no effect, since you cannot go move outside the beginning or end of the current line.

   For full marks you should not use indexing `(!!)` as this is inefficient (and messy). A good way to solve the problem is to represent the current line being edited as two lists, the characters before the line, and the characters after it. Note that using the function `last` is also inefficient, so you should avoid that to get full marks.

7. *(6 points)* Here is some code to implement a login process:

```
logIn :: IO ()
logIn = do
  putStrLn "Enter password:"
  loop
  putStrLn "Password correct."

  where
    -- Use recursion for loop
    loop = do
      guess <- getLine
      if guess /= "password123"
        then do
          putStrLn " Wrong password!"
          putStrLn " Try again:"
          loop
        else
          return ()
```

Ignoring the obvious security problems with this code, it would be nicer if we could write this in a more natural imperative style using a while-loop rather than recursion. Fortunately, in Haskell you can make your own control operators.

Give the definition for a general monadic while loop operation

```
whileM_ :: Monad m => m Bool -> m a -> m ()
```

so that the code above can be rewritten equivalently as

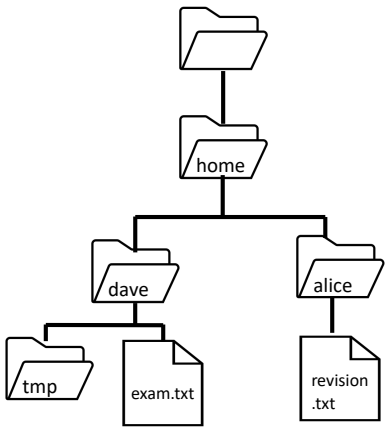
```
logIn' :: IO ()
logIn' = do
  putStrLn "Enter password:"
  whileM_ ((/= "password123") <$> getLine) $ do
    putStrLn " Wrong password!"
    putStrLn " Try again:"
  putStrLn "Password correct."
```

8. *(8 points)* The following data type is used to represent basic information about the structure of a computer file system:

```
type FileName = String
type DirName  = String

type FileSystem = [DirItem]
data DirItem = File FileName | Dir DirName FileSystem
```

For example, the file system below can be represented with the definition to the right:




```
exampleFS = [
              Dir "home"
                   [
                    Dir "alice"
                         [
                          File "revision.txt"
                         ]
                   ,Dir "dave"
                         [
                          File "exam.txt"
                         ,Dir "tmp" []
                         ]
                   ]
             ]
```

In this representation we assume that the order in which the directory items are listed is unimportant. An alternative way to represent such a filesystem is to use the following type

```
type AltFileSystem = [(Path,[FileName])]
type Path          = [DirName]
```

The alternative representation of the example above is:

```
exampleAltFS = [([]                   , []             )
               ,(["home"]             , []             )
               ,(["home","dave"]      , ["exam.txt"]   )
               ,(["home","dave","tmp"], []             )
               ,(["home","alice"]     , ["revision.txt"])]
```

Again, we assume that the order in which items are listed is not important. Note that every valid directory path in `exampleFS` is represented in `exampleAltFS`.

Write a function

```
checkFS :: FileSystem -> AltFileSystem -> Bool
```

which checks that the two representations are equivalent with no missing or extra information, so that for example the following is True:

```
prop_checkFS = checkFS exampleFS exampleAltFS
```

```
{- Some standard functions from Prelude Data.List
   Data.Maybe Data.Char Control.Monad -}
-----------------------------------------------
class Show a where
  show :: a -> String

class Eq a where
  (==), (/=) :: a -> a -> Bool

class (Eq a) => Ord a where
  (<), (<=), (>=), (>) :: a -> a -> Bool
  max, min             :: a -> a -> a

class (Eq a, Show a) => Num a where
  (+), (-), (*)        :: a -> a -> a
  negate, abs, signum  :: a -> a
  fromInteger          :: Integer -> a

class (Num a, Ord a) => Real a where
  toRational       ::  a -> Rational

class (Real a, Enum a) => Integral a where
  quot,rem,div,mod :: a -> a -> a
  toInteger        :: a -> Integer

class (Num a) => Fractional a where
  (/)              :: a -> a -> a
  fromRational     :: Rational -> a

class (Fractional a) => Floating a where
  exp, log, sqrt, sin, cos, tan :: a -> a

class (Real a, Fractional a) => RealFrac a where
  truncate, round, ceiling, floor
     :: (Integral b) => a -> b
-- numerical functions ---------------------------
even, odd        :: (Integral a) => a -> Bool
even n           = n `rem` 2 == 0
odd              = not . even

-- monadic functions -----------------------------
sequence     :: Monad m => [m a] -> m [a]
sequence     = foldr mcons (return [])
  where mcons p q = do x <- p
                       xs <- q
                       return (x:xs)

sequence_    :: Monad m => [m a] -> m ()
sequence_ xs = sequence xs >> return ()

-- functions on functions ------------------------
id               :: a -> a
id x             = x
const            :: a -> b -> a
const x _        = x

(.)              :: (b -> c) -> (a -> b) -> a -> c
f . g            = \ x -> f (g x)

flip             :: (a -> b -> c) -> b -> a -> c
flip f x y       = f y x

($)              :: (a -> b) -> a -> b
f $  x           = f x

----- functions on Bool -------------------------
(&&), (||)       :: Bool -> Bool -> Bool
True  && x       = x
False && _       = False
True  || _       = True
False || x       = x

not              :: Bool -> Bool
not True         = False
not False        = True

--- functions on Maybe -----------------------------
isJust, isNothing      :: Maybe a -> Bool
isJust (Just a)        =  True
isJust Nothing         =  False
isNothing              =  not . isJust

fromJust               :: Maybe a -> a
fromJust (Just a)      =  a

maybeToList            :: Maybe a -> [a]
maybeToList Nothing    =  []
maybeToList (Just a)   =  [a]
```

```
listToMaybe            :: [a] -> Maybe a
listToMaybe []         =  Nothing
listToMaybe (a:_)      =  Just a

catMaybes              :: [Maybe a] -> [a]
catMaybes ls           = [x | Just x <- ls]

-- functions on pairs ----------------------------
fst                    :: (a,b) -> a
fst (x,y)              =  x

snd                    :: (a,b) -> b
snd (x,y)              =  y

swap                   :: (a,b) -> (b,a)
swap (a,b)             = (b,a)

curry  :: ((a, b) -> c) -> a -> b -> c
curry f x y            =  f (x, y)

uncurry :: (a -> b -> c) -> ((a, b) -> c)
uncurry f p    =  f (fst p) (snd p)

-- functions on lists ----------------------------

map :: (a -> b) -> [a] -> [b]
map f xs = [ f x | x <- xs ]

(++) :: [a] -> [a] -> [a]
xs ++ ys = foldr (:) ys xs

filter :: (a -> Bool) -> [a] -> [a]
filter p xs = [ x | x <- xs, p x ]

concat :: [[a]] -> [a]
concat xss = foldr (++) [] xss

concatMap :: (a -> [b]) -> [a] -> [b]
concatMap f = concat . map f

head, last             :: [a] -> a
head (x:_)             = x

last [x]               = x
last (_:xs)            = last xs

tail, init             :: [a] -> [a]
tail (_:xs)            = xs

init [x]               = []
init (x:xs)            = x : init xs

null                   :: [a] -> Bool
null []                = True
null (_:_)             = False

length                 :: [a] -> Int
length                 = foldr (const (1+)) 0

(!!)                   :: [a] -> Int -> a
(x:_)  !! 0            = x
(_:xs) !! n            = xs !! (n-1)

foldr    :: (a -> b -> b) -> b -> [a] -> b
foldr f z []           =  z
foldr f z (x:xs)       =  f x (foldr f z xs)

foldl    :: (a -> b -> a) -> a -> [b] -> a
foldl f z []           =  z
foldl f z (x:xs)       =  foldl f (f z x) xs

iterate                :: (a -> a) -> a -> [a]
iterate f x            =  x : iterate f (f x)

repeat                 :: a -> [a]
repeat x               =  xs where xs = x:xs

replicate              :: Int -> a -> [a]
replicate n x          =  take n (repeat x)

cycle                  :: [a] -> [a]
cycle []               =  error "cycle: empty list"
cycle xs               =  xs' where xs' = xs ++ xs'

tails                  :: [a] -> [[a]]
tails xs               =  xs : case xs of
                                 []      -> []
                                 _ : xs' -> tails xs'
```

```
take, drop              :: Int -> [a] -> [a]
take n _       | n <= 0 =  []
take _ []               =  []
take n (x:xs)           =  x : take (n-1) xs

drop n xs      | n <= 0 =  xs
drop _ []               =  []
drop n (_:xs)           =  drop (n-1) xs

splitAt                 :: Int -> [a] -> ([a],[a])
splitAt n xs            =  (take n xs, drop n xs)

takeWhile, dropWhile :: (a -> Bool) -> [a] -> [a]
takeWhile p []                  = []
takeWhile p (x:xs) | p x        = x : takeWhile p xs
                   | otherwise  = []

dropWhile p []                  = []
dropWhile p (x:xs) | p x        = dropWhile p xs
                   | otherwise  = x:xs

span :: (a -> Bool) -> [a] -> ([a], [a])
span p as = (takeWhile p as, dropWhile p as)

lines, words     :: String -> [String]
-- lines "apa\nbepa\ncepa\n" ==["apa","bepa","cepa"]
-- words "apa  bepa\n cepa" == ["apa","bepa","cepa"]

unlines, unwords :: [String] -> String
-- unlines ["ap","bep","cep"] == "ap\nbep\ncep"
-- unwords ["ap","bep","cep"] == "ap bep cep"

reverse          :: [a] -> [a]
reverse          =  foldl (flip (:)) []

and, or          :: [Bool] -> Bool
and              =  foldr (&&) True
or               =  foldr (||) False

any, all         :: (a -> Bool) -> [a] -> Bool
any p            =  or . map p
all p            =  and . map p

elem, notElem    :: (Eq a) => a -> [a] -> Bool
elem x           =  any (== x)
notElem x        =  all (/= x)

lookup      :: (Eq a) => a -> [(a,b)] -> Maybe b
lookup key []    =  Nothing
lookup key ((x,y):xys) | key == x   = Just y
                       | otherwise  = lookup key xys

sum, product     :: (Num a) => [a] -> a
sum              =  foldl (+) 0
product          =  foldl (*) 1

maximum, minimum :: (Ord a) => [a] -> a
maximum [] = error "Prelude.maximum: empty list"
maximum (x:xs) = foldl max x xs

minimum [] = error "Prelude.minimum: empty list"
minimum (x:xs) = foldl min x xs

zip              :: [a] -> [b] -> [(a,b)]
zip              =  zipWith (,)

zipWith          :: (a->b->c) -> [a]->[b]->[c]
zipWith z (a:as) (b:bs)
                 =  z a b : zipWith z as bs
zipWith _ _ _    =  []

unzip            :: [(a,b)] -> ([a],[b])
unzip            =
  foldr (\(a,b) ~(as,bs) -> (a:as,b:bs)) ([],[])

nub              :: Eq a => [a] -> [a]
nub []           = []
nub (x:xs)       = x : nub [ y | y <- xs, x /= y ]

delete           :: Eq a => a -> [a] -> [a]
delete y []      = []
delete y (x:xs)  =
  if x == y then xs else x : delete y xs

(\\)             :: Eq a => [a] -> [a] -> [a]
(\\)             = foldl (flip delete)

union            :: Eq a => [a] -> [a] -> [a]
union xs ys      = xs ++ (ys \\ xs)
```

```
intersect        :: Eq a => [a] -> [a] -> [a]
intersect xs ys  = [ x | x <- xs, x `elem` ys ]

intersperse      :: a -> [a] -> [a]
-- intersperse 0 [1,2,3,4] == [1,0,2,0,3,0,4]

transpose        :: [[a]] -> [[a]]
-- transpose [[1,2,3],[4,5,6]] ==[[1,4],[2,5],[3,6]]

partition   :: (a -> Bool) -> [a] -> ([a],[a])
partition p xs = (filter p xs, filter (not . p) xs)

group            :: Eq a => [a] -> [[a]]
group = groupBy (==)

groupBy :: (a -> a -> Bool) -> [a] -> [[a]]
groupBy _  []     =  []
groupBy eq (x:xs) =  (x:ys) : groupBy eq zs
             where (ys,zs) = span (eq x) xs

isPrefixOf :: Eq a => [a] -> [a] -> Bool
isPrefixOf []     _      = True
isPrefixOf _      []     = False
isPrefixOf (x:xs) (y:ys) = x==y && isPrefixOf xs ys

isSuffixOf :: Eq a => [a] -> [a] -> Bool
isSuffixOf x y = reverse x `isPrefixOf` reverse y

sort             :: (Ord a) => [a] -> [a]
sort             = foldr insert []

insert           :: (Ord a) => a -> [a] -> [a]
insert x []      = [x]
insert x (y:xs)  =
    if x <= y then x:y:xs else y:insert x xs

-- functions on Char ----------------------------
type String = [Char]

isSpace, isDigit :: Char -> Bool
toUpper, toLower :: Char -> Char

digitToInt :: Char -> Int
-- digitToInt '8' == 8

intToDigit :: Int -> Char
-- intToDigit 3 == '3'

ord :: Char -> Int
chr :: Int  -> Char
-----------------------------------------
-- Useful functions from Test.QuickCheck
arbitrary :: Arbitrary a => Gen a
-- generator used by quickCheck

choose :: Random a => (a, a) -> Gen a
-- a random element in the given inclusive range.

oneof :: [Gen a] -> Gen a
-- Randomly uses one of the given generators
frequency :: [(Int, Gen a)] -> Gen a
-- Chooses from weighted list of generators

elements :: [a] -> Gen a
-- Generates one of the given values.

listOf :: Gen a -> Gen [a]
-- Generates a list of random length.
vectorOf :: Int -> Gen a -> Gen [a]
-- Generates a list of the given length.

sized :: (Int -> Gen a) -> Gen a
-- construct generators that depend a size param.
```